



THE CLOUD & AI SUMMIT • 2026

Sept 30 - Oct 2, 2026

Ameristar • St. Charles, MO

Thank you to our **sponsors.**

PREMIUM EXHIBITORS



EXHIBITORS



EXPERIENCE SPONSORS



Spec-Driven Development with **GitHub Spec Kit**



From blank page to finished draft — building a book-writing app, one spec at a time.

Barret Blake

Architect, NimblePros

<https://barretblake.dev>

BlueSky: @barretblake.dev

<https://linkedin.com/in/barretblake>



WHO AM I

Barret Blake



Barret Blake

Architect · NimblePros



A few things about me

-  Microsoft MVP — .NET
-  Husband & Father
-  Coder, Blogger & Speaker
-  Mentor
-  Gamer
-  Model Railroader & LEGO Enthusiast
-  Buckeyes & Browns Fan

What We'll Cover Today



The Problem

Why “vibe coding” breaks down at scale



Spec-Driven Development

Specs as the source of truth for AI agents



Spec Kit in 2026

What the toolkit is — and what's new this year



The Workflow

Constitution → Specify → Plan → Implement



Meet InkWell

Our demo: a novelist's writing studio



Make It Yours

Extensions, presets & best practices

Vibe Coding: Staring at a Blank Page



What is vibe coding?

Tossing vague prompts at an AI coding assistant and accepting whatever it hands back — no plan, no spec, just vibes.

Like starting a novel with no outline, no characters, no ending in mind.

The result:



Unpredictable, inconsistent code quality



Features that miss the actual requirement



Hours lost debugging the AI's misunderstandings



Over-engineered solutions nobody asked for

We Treat AI Like a Search Engine



“The issue isn't the coding agent's ability, but our approach. We treat coding agents like search engines when we should treat them more like literal-minded pair programmers.”

— Den Delimarsky, former GitHub Principal Product Manager



The key insight

AI assistants need clear, structured context and specifications — not vague wishes. They excel at execution when you give them real direction.

What Is Spec-Driven Development?

An approach where specifications become the source of truth — guiding AI agents to build exactly what's needed through clear, structured, living requirements.



Specifications First

Define what you want before a line of code exists



Living Documents

Specs evolve continuously — not written once and forgotten



Human Review

Every phase needs explicit approval before moving on



Single Source of Truth

One spec guides both humans and AI agents



Docs Live With Code

Specs and implementation share the same repo

● THE TOOLKIT

GitHub Spec Kit

An open-source toolkit that brings Spec-Driven Development to life — a simple CLI plus structured templates that work with the agent you already use.

106K+

GitHub stars

30

Agent integrations

105

Community extensions

200+

Contributors



Open source & agent-agnostic
by design



A Python CLI plus battle-tested
templates



Drops into VS Code or your
favorite AI tool

First released Sept 2025 by GitHub — now a fast-moving community platform.

What's New in Spec Kit



The --integration system

The old --ai flags are gone. Pick your agent with --integration copilot | claude | codex | gemini ...



Extensions & presets

A whole ecosystem: 105 extensions & 22 presets let you add capabilities or reshape the workflow.



Self-management built in

specify self check and specify self upgrade keep the CLI current — no reinstall dance.



Skills mode + 30 agents

Install Spec Kit as agent skills (--skills), and switch freely across 30+ supported agents.

The Specify CLI & Project Structure



The Specify CLI

- Bootstraps your project scaffolding
- Downloads the official SDD templates
- Wires up your chosen AI agent
- Installs the /speckit.* slash commands

Your project after init

```
.specify/  
  memory/constitution.md  
  templates/ (spec, plan, tasks)  
  scripts/  
  extensions/  presets/  
specs/  
  001-manuscript-drafting/  
    spec.md  plan.md  tasks.md  
.github/  .claude/  (agent commands)
```

Getting Started in Two Steps

Prerequisites



uv (or pipx)



Python 3.11+



Git



An AI agent

1 Install the Specify CLI

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@vX.Y.Z
```

2 Initialize your project & pick your agent

```
specify init inkwell --integration claude
```

Then open your agent and the /speckit. commands are ready to go.*

● THE WORKFLOW

From Intent to Implementation

Seven steps take InkWell from a single sentence to shipped code:



Quality gates along the way: `/speckit.clarify` before planning, `/speckit.checklist` and `/speckit.analyze` before you implement.

● OUR RUNNING DEMO

Meet InkWell

A novelist's writing studio — the app we'll build end-to-end with Spec Kit.



Chapter drafting

Distraction-free editor with autosave



Character & plot tracker

Keep your cast and threads straight



Word-count goals

Daily targets and streaks that motivate



Local-first & private

Manuscripts stay on the writer's device

Same story, every phase: watch InkWell grow from a sentence into a working app.

Specification 2 – Adding reading time

From one sentence to a reviewed plan



Step by step through the process in real-ish time.

Set the Ground Rules



The foundation

A set of non-negotiable principles the agent must honor in every later phase — quality bars, tech constraints, UX and accessibility standards. Stored in `.specify/memory/constitution.md`.



InkWell's constitution (9 principles)

- Clean architecture — dependencies point inward, always
- Tests before implementation, for every feature
- WCAG 2.1 AA is a mandate, not an aspiration
- Local-first, encrypted at rest; nothing leaves without consent

Describe What to Build



The what & the why

Focus on outcomes, not tech. Capture goals, target users, core features, and success criteria. Spec Kit creates a feature branch (001-manuscript-drafting) and a spec.md full of user stories.



InkWell's spec prompt

“Build an app for novelists to draft chapters and organize them into a manuscript. Writers track characters and plot threads, set daily word-count goals, and write in a distraction-free editor. Everything works offline and saves locally.”

Close the Gaps Before You Plan



Structured Q&A

A sequential, coverage-based interview surfaces what the first draft missed and records answers in a Clarifications section. Run it before `/speckit.plan` to cut rework. (Formerly `/quizme`.)



What InkWell actually got asked

- How are inline images stored, given encrypted local-first?
- How does the editor present content in distraction-free mode?
- What counts as a word — prose only, or markdown syntax too?
- Export granularity: whole manuscript, per chapter, or both?

Choose the How



The technical plan

Now name your stack. The agent reads the spec + constitution and produces plan.md, research.md, data-model.md, contracts/, and quickstart.md — architecture, data, and APIs in detail.



InkWell's plan prompt

“Use .NET MAUI with a Clean Architecture design and a minimal library footprint. Store manuscripts in an encrypted local database — nothing is uploaded. Keep the editor framework-agnostic and fully keyboard-navigable.” → the agent chose SQLite + SQLCipher (AES-256), key in SecureStorage. You never named it.

Unit Tests for Your English

Two optional commands catch problems while they're still cheap — before any code is written.



`/speckit.checklist`

Generate custom quality checklists that validate your requirements for completeness, clarity, and consistency — “unit tests for English.”

*For InkWell: does every user story define success?
Are offline behaviors specified?*



`/speckit.analyze`

Cross-artifact consistency and coverage analysis. Run it after `/speckit.tasks` and before `/speckit.implement` to catch drift between spec, plan, and tasks.

For InkWell: does the plan actually cover the export formats the spec promised?

Wishes vs. Specs: Writing for a Literal-Minded Reader

A WISH

✗ *"The editor should be fast."*

✗ *"Word counts should be accurate."*

✗ *"It works offline."*

✗ *"Never lose my work."*



A SPEC · WORD FOR WORD FROM INKWELL'S `spec.md`



Enter and exit distraction-free mode, back at the exact cursor position, in under 1 second.

SC-006



Matches an independent prose-word count — excluding markdown syntax and image markers — 100% of the time.

SC-005



Every capability works with networking fully disabled; 100% of user stories complete offline.

SC-002



No edit lost beyond the last few seconds of typing; restored on reopen in 100% of test cases.

SC-003



The test: could two reasonable people disagree about whether it passed?

My own SC-004 says typing must be “perceived as instantaneous.” The plan quietly made it 16 ms. **Leave a wish in the spec and the agent picks the number for you.**

Break It Into Actionable Work

/speckit.tasks



From plan to a build list

- Tasks grouped by user story — each ships independently
- Ordered by dependency (models → services → endpoints)
- Parallel-safe work flagged with [P]
- Exact file paths on every task
- Test-first ordering when TDD is requested



Bonus: /speckit.taskstoissues

Turn the generated task list into tracked GitHub issues in one command — so the whole team can pick up InkWell's work.

```
tasks.md
[P] T003  models/manuscript.ts
[P] T004  models/chapter.ts
      T007  editor/autosave.ts
      T012  export/epub.ts
```

```
/speckit.implement
```

Let the Agent Build InkWell



Validates prerequisites

Constitution, spec, plan, and tasks all present



Parses tasks.md

Reads the ordered breakdown and dependencies



Executes in order

Respects sequence and [P] parallel markers



Follows TDD & the plan

Tests first where you asked; principles enforced



The result: a working InkWell — chapters, characters, goals, and a distraction-free editor — built from specs, not vibes.

Review, Verify, Merge



Human oversight is the point

Spec Kit gets you a working draft fast — but you're still the author. Read the code, run it, catch what CLI logs won't show (browser console errors, UX rough edges), and paste issues back to the agent.

1

Review the diff

Does the code match the spec and constitution?

2

Run & QA

Exercise InkWell — write a chapter, hit a word goal

3

Sign off

Approve, open a PR with a clear description

4

Merge

Ship it — with docs living beside the code

● STEP 7, HONESTLY

Reading the Diff: Five Smells

1

Code that contradicts a numbered requirement

REAL · INKWELL

FR-004: “without requiring an explicit save action.” Three screens shipped with Save buttons.

2

Green tests, broken app

REAL · INKWELL

Chapter text never saved. The tests substitute the editor — so every one of them passed.

3

Tests that changed with the code

An assertion loosened in the same diff as the fix it is supposed to verify.

4

A gate that graded itself

Constitution Check: 9 of 9 PASS — written by the author of the plan it is passing.

5

Work outside the lines

Files the task never named. A package the plan never chose.



Smell #2, in three lines

```
function reportContent(type) {  
  if (!view || !chapterId)  
    return; // every keystroke  
}
```

chapterId was only ever set by a message that got dropped on the floor. No error. No failing test. No saved chapters.

specs/001-.../research.md \$5.4

When It Changes: Fix It at the Right Altitude

EACH LAYER IS GENERATED FROM THE ONES ABOVE IT



⚠ **The hotfix trap:** patch the code, leave the spec — and the next regenerate faithfully reintroduces the bug you fixed.

What Actually Gets Generated



From one sentence and four prompts

- 13 files. ~1,700 lines.
- Numbered requirements and success criteria
- The alternatives the agent considered — and rejected
- Interface contracts, traced to requirement IDs

All of it plain markdown, in git, reviewable

```
specs/001-manuscript-drafting/  
  spec.md                235 lines  
  research.md            300 lines  
  data-model.md          163 lines  
  extensions/            presets/  
  contracts/              5 files  
  quickstart.md          122 lines  
  checklists/             requirements.md  
  tasks.md                439 lines
```

Anatomy of a Generated Contract



**37 lines, written during
/speckit.plan**

It knows which requirements it serves, names the domain types it needs, specifies their behaviour precisely enough to implement from — and states its own test obligations, before any code exists.



Straight from the repo

“Serves: User Story 3 (P2) · FR-009...FR-012” ·
ProseWordCounter, DailyProgressCalculator, GoalEvaluator
+ IClock injected for testability · “Counts tokens only within
literal prose inline text — excludes syntax, links, image
markers.” · “Contract tests (unit, TDD): counts match an
independent prose count...” → that last line is constitution
§II becoming an instruction the agent follows three phases
later.

Make Spec Kit Your Own



Extensions

ADD new capabilities

New commands, phases, and integrations. CI Guard, Architecture Guard, Jira sync — or an InkWell “manuscript-export” command.

```
specify extension add <name>
```



Presets

CHANGE how it works

Reshape templates, terminology, and standards without new commands. Enforce a house style guide on every spec and plan.

```
specify preset add <name>
```



Or swap the whole process: AIDE, Canon, Product Forge, MAQA — community presets that redefine SDD end-to-end. 105 extensions, 22 presets, and counting.

Make It Yours

No model involved



CLI only



Fast



Deterministic

1 Add an extension

```
specify extension list
specify extension search
specify extension add <name>
```

2 Add a preset — the templates rewrite themselves

```
specify preset <command>
```

Encode your architecture rules, your definition of done, your compliance language once — and every spec your team generates comes out already shaped that way. Extensions are also where governance lives: CI Guard and Architecture Guard add gates, not memos.

Works With 30+ AI Agents

Spec Kit is agent-agnostic. Switch freely with one flag — run `specify integration list` to see them all.



CLI / Terminal

Claude Code
Codex CLI
Gemini CLI
Qwen Code
opencode
Goose
Mistral Vibe
Forge



IDE Assistants

GitHub Copilot
Cursor
Windsurf
Kiro
Tabnine
Qoder
Roo Code
Trae



Escape Hatch

Generic integration —
bring any agent

Skills mode:

`--integration-
options="--skills"`
installs agent skills

Best Practices & Tips



Do

- ✓ Start with the MVP — one small spec first
- ✓ Review each phase before moving on
- ✓ Use specific, measurable success criteria
- ✓ Keep a human supervising the agent
- ✓ Iterate — specs are living documents



Don't

- ✗ Skip the constitution — it's the foundation
- ✗ Let the AI run unsupervised
- ✗ Treat specs as fixed waterfall documents
- ✗ Over-engineer from the very start
- ✗ Cram everything into one giant spec

When NOT to Use Spec-Driven Development



Throwaway work

Run-once scripts, data fixes, disposable spikes



Genuine exploration

When you don't know what you want, a spec is a lie



Very small, known changes

Adding a field. The overhead exceeds the risk removed.



Nobody reading the output

Rubber-stamping is vibe coding with extra latency

Small and disposable, vibe it. Unknown and exploratory, prototype it. Real, shared, outliving the sprint? Spec it.

Starting on the Codebase You Already Have

1 `$ specify init --here --integration copilot` Adds .specify/ and your agent's command files. Touches none of your code.

2 **Write the constitution from what is already true**

Let the agent read the repo and draft it; you edit. Rules you'll enforce on the next pull request — not the architecture you wish you had.

3 **Don't retro-spec the system**

Your first spec is your next feature. Specs accumulate forward — 001, 002 — so the code that changes most ends up documented best.

4 **Pick a first feature with a seam**

Small, additive, one area of the code. Reading time — not “rewrite the billing engine.”

5 **Point the plan at the code that is there**

“...following the existing clean architecture.” The plan phase reads the repo; your patterns become the path of least resistance.

No tests around the code you're about to touch? Then your first spec is characterization tests — and that is a perfectly good first feature.

one team, one real feature

Making It Stick on a Team



The constitution becomes org policy

Standards move from a wiki nobody opens into a file the generator obeys



Specs get reviewed like code

Review the spec before the implementation exists — when changing your mind is free



Quality gates move into CI

/speckit.analyze as a pipeline step; CI Guard and Architecture Guard as gates



The task list becomes the board

/speckit.taskstoissues — tracked issues with dependencies intact



Don't roll it out org-wide: pick one team and one real feature, and let the artifacts be the argument. A spec folder someone can open beats any deck about methodology — including this one.

The Spec-Driven Ecosystem

Spec Kit isn't alone — structured, spec-first development is becoming the default for AI-assisted work.



Amazon Kiro

Agentic IDE with spec-driven development built in



Tessl Framework

A spec-driven registry for reliable AI coding agents



Alt SDD processes

AIDE, Canon & Product Forge reimagine the workflow



MAQA

Multi-agent orchestration with QA gates

Five Things to Remember

1

Vibe coding doesn't scale

Clear specs are essential for reliable AI-assisted development

2

Spec Kit gives you structure

A proven path from constitution to implementation — and beyond

3

It's more powerful in 2026

Extensions, presets, and skills mode make it truly yours

4

Works with your tools

30+ agents, no lock-in — Copilot, Claude, Cursor, and more

5

You're still the author

Start small, iterate often, and keep a human in the loop

Thank You!

Questions?

GitHub Spec-kit github.com/github/spec-kit

BlueSky [@barretblake.dev](https://bsky.app/profile/barretblake.dev)

Web barretblake.dev

YouTube youtube.com/@barretcodes

LinkedIn linkedin.com/in/barretblake

NimblePros blog.nimblepros.com

Repo & Slides: github.com/barretb/inkwell



Session Feedback

